Lexical_2

- Read&understand the **specification of the WHILE language (https://canvas.elte.hu/courses/65695/pages/language-specification-while)**
- Download the **lexical analyzer for the WHILE language (https://canvas.elte.hu/courses/65695/files/4479678/download?wrap=1)** , try to understand the source code
- Try it out on your local lab computer -- WORKS ONLY WITH LINUX
  - You can also use pandora.inf.elte.hu; copy files using Winscp.
- Try to implement&extend the WHILE lexical analyzer with a few language extensions from **1. week (https://canvas.elte.hu/courses/65695/pages/lexical-1)**
- Let's extend our lexical analyzer with the following new language features - **test files (https://canvas.elte.hu/courses/65695/files/4479595/download?wrap=1)** :
  - Identifier - change the T_ID rule:
    - the first part of the identifier can only contain letters and _ (underscore), while the second part can only contain digits.
    - if the second part - which can only contain digits - starts with an even digit, it is only accepted to have an even number of digits after it, if the first one is odd then it is only accepted to have an odd number of digits (so the restriction is not on the digits themselves but on their total number)
  - Number literals - add new rules besides T_NUM:
    - Binary literals begin with "0b" and contain binary digits (0 or 1).
    - Octal literals start with "0o" (zero o) and contain digits from 0 to 7.
    - Hexadecimal literals begin with "0x" and may contain digits and/or letters from a to f (letters may be capital).
    - For example, 0xaF or 0o7 are valid literals, but 0o9 and 0xx are not.
    - Senary literals (base 14): starts with "14" and may contain digits and/or letters from a to d (letters may be capital as well)
      - Valid examples: 140, 14d3a9
      - Invalid examples: 14, 14e, 14444g, 1133
      - Please double-check whether your senary literal recognition really happens (keep in mind you have a general [0-9]+ like rule for number recognition as well)
    - Sexagesimal literals (base 60): starts with "60" and every 'digit' can take the following values 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13.....59 - so every 'digit' takes two characters => every valid sexagesimal number builds up from an even number of characters
      - every number that starts with 60 should automatically be considered as a sexagesimal instead of a decimal
      - for example, 600223 is a valid sexagesimal; it is actually the 0223 sexagesimal number (which is 143 in decimal numbers)

        - more valid examples: 6000, 60475902
        - invalid examples: 6100, 60, 603479
        - Please double-check whether your sexagesimal literal recognition really happens (keep in mind you have a general [0-9]+ like rule for number recognition as well)
    - Multiline comment, starting with /* and ending with the first */
        - version1: the inside of the comment can only contain letters
        - version2: the inside of the comment can contain anything except the * (star) sign
        - version3: even the * (star) is accepted inside the comment (solution: "/*"([^*]+*|*)(([^/*][^*]**)|*)*"/")
        - **multi_comment.jpg (https://canvas.elte.hu/courses/65695/files/4479605/download?wrap=1)**
        - **multi_comment.ok (https://canvas.elte.hu/courses/65695/files/4479604/download?wrap=1)**
    - Introduce a string literal, which is a character sequence enclosed by double quotes. The string literal can contain anything except a double quote. However, the length of the string literal (without the double quotes) should be divisible by 3, so the 3,6,9,12 ... length should only be accepted
    - Introduce a date literal with the following format YY-MM-DD where the YY can be any future year (so 22,23,24 ... 99 are the acceptable values), MM can recognize only 01,02 .. 12, while DD runs from 01 to 31.
        - After creating the date literal recognition, let's create a rule that will recognize those "date"s that are formatted as YY-MM-DD, but violate the above-mentioned boundaries, meaning either the YY is below 22, or MM is bigger than 12 or DD is bigger than 31. For these print out an error message and exit from the lexical analyzer.